

# Python random Module

Brief Theory • Important Functions • Examples • Number Guessing Game

## 1. What is the random module?

The **random** module is a built-in Python module used to generate **pseudo-random** numbers and to make random selections. It is commonly used in games, simulations, testing, sampling, and programs where a random choice is required.

```
import random
```

## 2. Important Functions

| Function                            | Brief theory                                                                                                                                  |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>random()</b>                     | Returns a random floating-point number from <b>0.0</b> (inclusive) to <b>1.0</b> (exclusive).                                                 |
| <b>randint(a, b)</b>                | Returns a random integer from <b>a</b> to <b>b</b> . Both endpoints are included.                                                             |
| <b>randrange(start, stop, step)</b> | Returns a random integer from the sequence produced by Python's <b>range()</b> . The stop value is excluded and an optional step can be used. |
| <b>uniform(a, b)</b>                | Returns a random floating-point number between <b>a</b> and <b>b</b> . Useful when decimal values are required.                               |
| <b>choice(sequence)</b>             | Selects and returns one random element from a sequence such as a list, tuple, or string.                                                      |
| <b>choices(sequence, k=n)</b>       | Selects multiple random elements. Elements may repeat because selection is with replacement.                                                  |
| <b>sample(sequence, k=n)</b>        | Selects multiple unique elements. The same element is not selected twice.                                                                     |
| <b>shuffle(list)</b>                | Randomly rearranges the elements of a list <b>in place</b> . It returns <b>None</b> .                                                         |
| <b>seed(n)</b>                      | Sets the starting state of the random generator. Using the same seed produces the same sequence, which is useful for reproducible testing.    |

## 3. Quick Examples

**random.random()**

```
print(random.random())
```

**randint()**

```
print(random.randint(1, 10))
```

**randrange()**

```
print(random.randrange(0, 11, 2))
```

**uniform()**

```
print(random.uniform(1.5, 5.5))
```

### choice()

```
print(random.choice(['Red', 'Blue', 'Green']))
```

### choices()

```
print(random.choices([1, 2, 3, 4, 5], k=3))
```

### sample()

```
print(random.sample([1, 2, 3, 4, 5], k=3))
```

### shuffle()

```
numbers = [1, 2, 3, 4, 5]
random.shuffle(numbers)
print(numbers)
```

### seed()

```
random.seed(10)
print(random.randint(1, 100))
```

## 4. randint() vs randrange()

| Feature         | randint(a, b) | randrange(start, stop, step) |
|-----------------|---------------|------------------------------|
| Stop value      | Included      | Excluded                     |
| Step            | Not supported | Supported                    |
| Example         | randint(1, 5) | randrange(1, 5)              |
| Possible values | 1, 2, 3, 4, 5 | 1, 2, 3, 4                   |

**Memory trick:** randint → both ends included. randrange → behaves like range(), so stop is excluded and step is possible.

## 5. Simple Number Guessing Game

The computer chooses a random number, and the user keeps guessing until the correct number is entered.

```
import random

secret_number = random.randint(1, 100)

print("Guess the number between 1 and 100!")

while True:
    guess = int(input("Enter your guess: "))

    if guess < secret_number:
        print("Too low!")
    elif guess > secret_number:
        print("Too high!")
    else:
        print("Correct! You guessed the number.")
        break
```

### How the game works

1. **import random** loads the random module.
2. **randint(1, 100)** generates the secret number.

3. **while True** keeps the game running.
4. The user enters a guess.
5. **if / elif / else** compares the guess with the secret number.
6. **break** stops the loop when the guess is correct.

**Exam tip:** `randrange()` does not accept decimal arguments. For random decimal values, use `uniform()`.